CLEANING DATASET

In [1]:
```
import pandas as pd
import numpy as np
import re
import os
import json
import matplotlib.pyplot as plt
from sklearn.preprocessing import MultiLabelBinarizer

RAW_CSV = "Kelp4_dataset_1.csv"
CLEAN_CSV = "Kelp4_dataset_2.csv"

df_raw = pd.read_csv(RAW_CSV, encoding="utf-8-sig")

print("Jumlah data mentah:", len(df_raw))
print("Jumlah kolom:", len(df_raw.columns))
df_raw.head()
```

```
Jumlah data mentah: 2500
Jumlah kolom: 117
```

Out[1]:

	address	categories/0	categoryName	cid	city	countryCod
0	Jl. Colombo, Karang Malang, Caturtunggal, Kec....	Bakso restaurant	Bakso restaurant	11075477362009281968	Sleman Regency	ID
1	Jl. Colombo, Karang Malang, Caturtunggal, Kec....	Bakso restaurant	Bakso restaurant	11075477362009281968	Sleman Regency	ID
2	Jl. Colombo, Karang Malang, Caturtunggal, Kec....	Bakso restaurant	Bakso restaurant	11075477362009281968	Sleman Regency	ID
3	Jl. Colombo, Karang Malang, Caturtunggal, Kec....	Bakso restaurant	Bakso restaurant	11075477362009281968	Sleman Regency	ID
4	Jl. Colombo, Karang Malang, Caturtunggal, Kec....	Bakso restaurant	Bakso restaurant	11075477362009281968	Sleman Regency	ID

5 rows × 117 columns

In [2]:
```
df_raw[["reviewId", "title", "categoryName", "text", "publishedAtDate", "stars", "o
```

Out[2]:

	reviewId	title	categoryName	t
0	Ci9DQUIRQUNvZENodHljRjlvT2s1S1JHbFpIRkJoV0hKVV...	Bakso Pa'jero	Bakso restaurant	N
1	Ci9DQUIRQUNvZENodHljRjlvT21WTVZHbFJURWwxT1daaE...	Bakso Pa'jero	Bakso restaurant	standar~ bakso ngeba
2	Ci9DQUIRQUNvZENodHljRjlvT21SUFdHMWlVbEo0TlZkSF...	Bakso Pa'jero	Bakso restaurant	pesa lama ke
3	Ci9DQUIRQUNvZENodHljRjlvT205Q2NUUm9ZbE51UzFSR2...	Bakso Pa'jero	Bakso restaurant	N
4	Ci9DQUIRQUNvZENodHljRjlvT2xWdmRYY3RRa0V0YzFWdV...	Bakso Pa'jero	Bakso restaurant	N
5	Ci9DQUIRQUNvZENodHljRjlvT21oamJETmZOMEprZUhaMV...	Bakso Pa'jero	Bakso restaurant	Saya ke jam 3 s bersa kelua yg
6	Ci9DQUIRQUNvZENodHljRjlvT21wTk1XNU9aekV3YzNVeV...	Bakso Pa'jero	Bakso restaurant	Bakso ba ya enak
7	Ci9DQUIRQUNvZENodHljRjlvT21sck1VSmtTamx2VTBWSG...	Bakso Pa'jero	Bakso restaurant	Semua w ba moza e bar
8	Ci9DQUIRQUNvZENodHljRjlvT2pjdFExaGZURmRoY1V0dm...	Bakso Pa'jero	Bakso restaurant	Manda a swi
9	Ci9DQUIRQUNvZENodHljRjlvT2tKM1EwNUNWRU5XU1hsRm...	Bakso Pa'jero	Bakso restaurant	N

In [3]:

```
def clean_review(text):
    if pd.isna(text):
        return ""

    text = str(text)

    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"[\n\r\t]", " ", text)
    text = re.sub(r"~+", " ", text)
    text = re.sub(r"\s+", " ", text)

    text = text.strip()
```

```
    return text
```

In [4]:

```
df_clean = pd.DataFrame()

df_clean["review_id"] = df_raw["reviewId"]
df_clean["business_name"] = df_raw["title"]
df_clean["category"] = "Kuliner"
df_clean["review_text"] = df_raw["text"].apply(clean_review)

review_date = pd.to_datetime(df_raw["publishedAtDate"], errors="coerce", utc=True)
df_clean["review_date"] = review_date.dt.tz_convert("Asia/Jakarta").dt.strftime("%d

df_clean["rating"] = df_raw["stars"]
df_clean["source_url"] = df_raw["reviewUrl"].fillna(df_raw["url"])
df_clean["place_id"] = df_raw["placeId"]
df_clean["address"] = df_raw["address"]
df_clean["city"] = df_raw["city"]
df_clean["original_language"] = df_raw["originalLanguage"]

df_clean.head()
```

Out[4]:

	review_id	business_name	category	review
0	Ci9DQUlRQUNvZENodHljRjlvT2s1S1JHbFplRkJoV0hKVV...	Bakso Pa'jero	Kuliner	
1	Ci9DQUlRQUNvZENodHljRjlvT21WTVZHbFJURWwxT1daaE...	Bakso Pa'jero	Kuliner	sta baks ngeb
2	Ci9DQUlRQUNvZENodHljRjlvT21SUFdHMWlVbEo0TlZkSF...	Bakso Pa'jero	Kuliner	pes lama k
3	Ci9DQUlRQUNvZENodHljRjlvT205Q2NUUm9ZbE51UzFSR2...	Bakso Pa'jero	Kuliner	
4	Ci9DQUlRQUNvZENodHljRjlvT2xWdmRYY3RRa0V0YzFWdV...	Bakso Pa'jero	Kuliner	

In [5]:
```
print("Jumlah awal:", len(df_clean))

before = len(df_clean)
df_clean = df_clean[df_clean["review_text"].notna()]
df_clean = df_clean[df_clean["review_text"].str.strip() != ""]
after = len(df_clean)

print("Setelah hapus review kosong:", after)
print("Review kosong yang dihapus:", before - after)
```

```
Jumlah awal: 2500
Setelah hapus review kosong: 1829
Review kosong yang dihapus: 671
```

In [6]:
```
before = len(df_clean)

df_clean = df_clean[df_clean["original_language"] == "id"]

after = len(df_clean)

print("Setelah hapus review bahasa Inggris/selain Indonesia:", after)
print("Data non-Indonesia yang dihapus:", before - after)
```

```
Setelah hapus review bahasa Inggris/selain Indonesia: 1707
Data non-Indonesia yang dihapus: 122
```

In [7]:
```
df_clean["word_count"] = df_clean["review_text"].apply(lambda x: len(str(x).split()

before = len(df_clean)

df_clean = df_clean[df_clean["word_count"] >= 5]

after = len(df_clean)

print("Setelah hapus review kurang dari 5 kata:", after)
print("Review kurang dari 5 kata yang dihapus:", before - after)
```

```
Setelah hapus review kurang dari 5 kata: 1227
Review kurang dari 5 kata yang dihapus: 480
```

In [8]:
```
before = len(df_clean)

df_clean = df_clean.drop_duplicates(subset=["review_text"])
df_clean = df_clean.drop_duplicates(subset=["review_id"])

after = len(df_clean)

print("Setelah deduplikasi:", after)
print("Duplikat yang dihapus:", before - after)
```

```
Setelah deduplikasi: 1227
Duplikat yang dihapus: 0
```

In [9]:
```
df_clean = df_clean.reset_index(drop=True)

df_clean = df_clean[
    [
```

```
        "review_id",
        "business_name",
        "category",
        "review_text",
        "review_date",
        "rating",
        "source_url",
        "place_id",
        "address",
        "city"
    ]
]

df_clean.head()
```

Out[9]:

	review_id	business_name	category	review
0	Ci9DQUIRQUNvZENodHljRjlvT21oamJETmZOMEprZUhaMV...	Bakso Pa'jero	Kuliner	Saya l jam 3 ber keluarg
1	Ci9DQUIRQUNvZENodHljRjlvT21wTk1XNU9aekV3YzNVeV...	Bakso Pa'jero	Kuliner	Bakso l ya enal
2	Ci9DQUIRQUNvZENodHljRjlvT21sck1VSmtTamx2VTBWSG...	Bakso Pa'jero	Kuliner	Semu mo ba
3	Ci9DQUIRQUNvZENodHljRjlvT201clVXVlZUekZDUmt0RF...	Bakso Pa'jero	Kuliner	Ud 3: tiap k baks enak.
4	Ci9DQUIRQUNvZENodHljRjlvT2s1b1p6VkRWR3R0VTJ0Q0...	Bakso Pa'jero	Kuliner	serv oke ba da pela

In [10]:

```
print("Jumlah dataset bersih:", len(df_clean))
print("Jumlah duplikat review_id:", df_clean["review_id"].duplicated().sum())
print("Jumlah duplikat review_text:", df_clean["review_text"].duplicated().sum())
print("Jumlah review kosong:", df_clean["review_text"].isna().sum())

df_clean["review_text"].apply(lambda x: len(str(x).split())).describe()
```

```
Jumlah dataset bersih: 1227
Jumlah duplikat review_id: 0
Jumlah duplikat review_text: 0
Jumlah review kosong: 0
```

Out[10]:
```
count    1227.000000
mean       22.757946
std        29.203330
min         5.000000
25%         7.000000
50%        12.000000
75%        24.500000
max       276.000000
Name: review_text, dtype: float64
```

In [11]:
```
df_clean.to_csv(CLEAN_CSV, index=False, sep=";", encoding="utf-8-sig")

print("Dataset bersih berhasil disimpan sebagai:", CLEAN_CSV)
```

```
Dataset bersih berhasil disimpan sebagai: Kelp4_dataset_2.csv
```

EDA DATASET HASIL ANOTASI

In [12]:
```
# ========================
# 1. Import Library
# ========================

import json
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MultiLabelBinarizer
```

In [13]:
```
# ========================
# 2. Load Dataset Anotasi
# ========================

ANNOTATION_JSONL = "Kelp4_dataset_anotasi.jsonl"

data = []

with open(ANNOTATION_JSONL, "r", encoding="utf-8") as f:
    for line in f:
        data.append(json.loads(line))

df_annot = pd.DataFrame(data)

print("Jumlah baris anotasi:", len(df_annot))
print("Jumlah kolom:", len(df_annot.columns))
print("Nama kolom:")
print(df_annot.columns.tolist())

df_annot.head()
```

```
Jumlah baris anotasi: 2118
Jumlah kolom: 13
Nama kolom:
['text', 'tokens', 'spans', '_input_hash', '_task_hash', 'options', '_view_id', 'acc
ept', 'config', 'answer', '_timestamp', '_annotator_id', '_session_id']
```

Out[13]:

	text	tokens	spans	_input_hash	_task_hash	options	_vi
0	Semuanya worth bakso mozanya enak banget	[{'text': 'Semuanya', 'start': 0, 'end': 8, 'i...	[{'start': 0, 'end': 40, 'token_start': 0, 'to...	-832848900	-485476066	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
1	Bakso bakar ya enak dan pas	[{'text': 'Bakso', 'start': 0, 'end': 5, 'id':...	[{'start': 0, 'end': 27, 'token_start': 0, 'to...	616756199	-494867163	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
2	Saya kesini jam 3 sore bersama keluarga, yg sa...	[{'text': 'Saya', 'start': 0, 'end': 4, 'id':	[{'start': 41, 'end': 75, 'token_start': 8, 't...	-1099850682	-1708591340	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
3	Ud 3x ksni tiap k jogja, baksonya enak, saran ...	[{'text': 'Ud', 'start': 0, 'end': 2, 'id': 0,...	[{'start': 0, 'end': 23, 'token_start': 0, 'to...	1518700474	1889327892	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
4	servisnya oke banget, baru dateng, mba pelayan...	[{'text': 'servisnya', 'start': 0, 'end': 9, '...	[{'start': 0, 'end': 20, 'token_start': 0, 'to...	1104821067	-1778492930	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E

In [14]:

```python
# ========================
# 3. Cek Jumlah Annotator dan Review Unik
# ========================

if "_annotator_id" in df_annot.columns:
    print("Jumlah data per annotator:")
    print(df_annot["_annotator_id"].value_counts())

if "_input_hash" in df_annot.columns:
    print("\nJumlah review unik:", df_annot["_input_hash"].nunique())
```

```
Jumlah data per annotator:
_annotator_id
nlp4_genap2526-230712427    1232
nlp4_genap2526-230712245     886
Name: count, dtype: int64

Jumlah review unik: 1232
```

In [15]:

```python
# ========================
# 4. Ambil Satu Anotasi per Review untuk EDA
# ========================
# Karena file anotasi berisi hasil dari lebih dari satu annotator,
# maka EDA memakai satu annotator dengan jumlah data paling banyak
# agar review yang sama tidak terhitung dua kali.

df_eda = df_annot.copy()

if "_annotator_id" in df_eda.columns and "_input_hash" in df_eda.columns:
    main_annotator = df_eda["_annotator_id"].value_counts().idxmax()

    df_eda = df_eda[df_eda["_annotator_id"] == main_annotator].copy()
    df_eda = df_eda.drop_duplicates(subset=["_input_hash"])

    print("Annotator yang digunakan untuk EDA:", main_annotator)
    print("Jumlah data EDA:", len(df_eda))
else:
    print("Kolom annotator atau input_hash tidak ditemukan.")
    print("Semua baris digunakan untuk EDA.")
    print("Jumlah data EDA:", len(df_eda))

df_eda.head()
```

```
Annotator yang digunakan untuk EDA: nlp4_genap2526-230712427
Jumlah data EDA: 1232
```

Out[15]:

	text	tokens	spans	_input_hash	task_hash	options	vi
0	Semuanya worth bakso mozanya enak banget	[{'text': 'Semuanya', 'start': 0, 'end': 8, 'i...	[{'start': 0, 'end': 40, 'token_start': 0, 'to...	-832848900	-485476066	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
1	Bakso bakar ya enak dan pas	[{'text': 'Bakso', 'start': 0, 'end': 5, 'id':...	[{'start': 0, 'end': 27, 'token_start': 0, 'to...	616756199	-494867163	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
2	Saya kesini jam 3 sore bersama keluarga, yg sa...	[{'text': 'Saya', 'start': 0, 'end': 4, 'id': ...	[{'start': 41, 'end': 75, 'token_start': 8, 't...	-1099850682	-1708591340	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
3	Ud 3x ksni tiap k jogja, baksonya enak, saran ...	[{'text': 'Ud', 'start': 0, 'end': 2, 'id': 0,...	[{'start': 0, 'end': 23, 'token_start': 0, 'to...	1518700474	1889327892	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E
4	servisnya oke banget, baru dateng, mba pelayan...	[{'text': 'servisnya', 'start': 0, 'end': 9, '...	[{'start': 0, 'end': 20, 'token_start': 0, 'to...	1104821067	-1778492930	[{'id': 'PRODUCT_POSITIVE', 'text': 'PRODUCT_P...	E

Distribusi Label ABSA

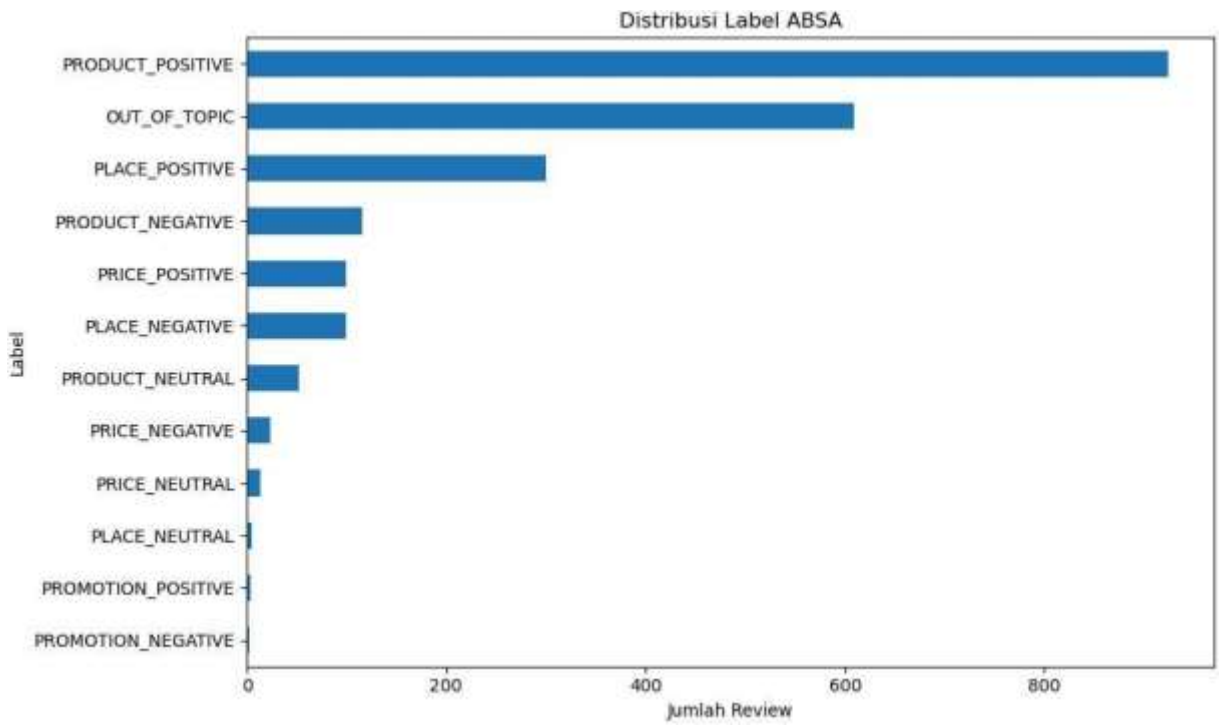
In [16]:

```
# ========================
# 5. Distribusi Label ABSA
# ========================

df_eda["accept"] = df_eda["accept"].apply(lambda x: x if isinstance(x, list) else [

label_counts = df_eda["accept"].explode().value_counts()

print("Distribusi Label ABSA:")
label_counts
```

```
Distribusi Label ABSA:
```

Out[16]:
```
accept
PRODUCT_POSITIVE      924
OUT_OF_TOPIC          609
PLACE_POSITIVE        300
PRODUCT_NEGATIVE      115
PLACE_NEGATIVE         99
PRICE_POSITIVE         99
PRODUCT_NEUTRAL        52
PRICE_NEGATIVE         23
PRICE_NEUTRAL          13
PLACE_NEUTRAL           5
PROMOTION_POSITIVE      4
PROMOTION_NEGATIVE      2
Name: count, dtype: int64
```

In [17]:
```
# Visualisasi Distribusi Label ABSA

plt.figure(figsize=(10, 6))
label_counts.sort_values().plot(kind="barh")
plt.title("Distribusi Label ABSA")
plt.xlabel("Jumlah Review")
plt.ylabel("Label")
plt.tight_layout()
plt.show()
```



In [18]:
```
# Tabel Distribusi Label ABSA

label_count_df = label_counts.reset_index()
label_count_df.columns = ["label", "jumlah"]

label_count_df
```

Out[18]:

	label	jumlah
0	PRODUCT_POSITIVE	924
1	OUT_OF_TOPIC	609
2	PLACE_POSITIVE	300
3	PRODUCT_NEGATIVE	115
4	PLACE_NEGATIVE	99
5	PRICE_POSITIVE	99
6	PRODUCT_NEUTRAL	52
7	PRICE_NEGATIVE	23
8	PRICE_NEUTRAL	13
9	PLACE_NEUTRAL	5
10	PROMOTION_POSITIVE	4
11	PROMOTION_NEGATIVE	2

Distribusi Entitas ABSA

In [19]:

```
# ========================
# 6. Membuat DataFrame Entitas dari Spans
# ========================

entities = []

for _, row in df_eda.iterrows():
    text = row.get("text", "")
    spans = row.get("spans", [])

    if not isinstance(spans, list):
        continue

    for span in spans:
        start = span.get("start")
        end = span.get("end")
        label = span.get("label")

        if start is not None and end is not None:
            entity_text = text[start:end]

            entities.append({
                "entity_text": entity_text,
                "entity_label": label,
                "start": start,
                "end": end,
                "review_text": text
            })
```

```
entity_df = pd.DataFrame(entities)

print("Jumlah entitas ABSA:", len(entity_df))
entity_df.head()
```

```
Jumlah entitas ABSA: 2862
```

Out[19]:

	entity_text	entity_label	start	end	review_text
0	Semuanya worth bakso mozanya enak banget	PRODUCT_POSITIVE	0	40	Semuanya worth bakso mozanya enak banget
1	Bakso bakar ya enak dan pas	PRODUCT_POSITIVE	0	27	Bakso bakar ya enak dan pas
2	yg saya suka pelayanan disini oke,	OUT_OF_TOPIC	41	75	Saya kesini jam 3 sore bersama keluarga, yg sa...
3	semua pelayan ketika ada makanan yg kelewat at...	OUT_OF_TOPIC	76	163	Saya kesini jam 3 sore bersama keluarga, yg sa...
4	semua pelayanannya muslim	OUT_OF_TOPIC	164	189	Saya kesini jam 3 sore bersama keluarga, yg sa...

In [20]:

```
# ========================
# 7. Distribusi Entitas Berdasarkan Label
# ========================

entity_label_counts = entity_df["entity_label"].value_counts()

print("Distribusi Entitas ABSA:")
entity_label_counts
```

```
Distribusi Entitas ABSA:
```

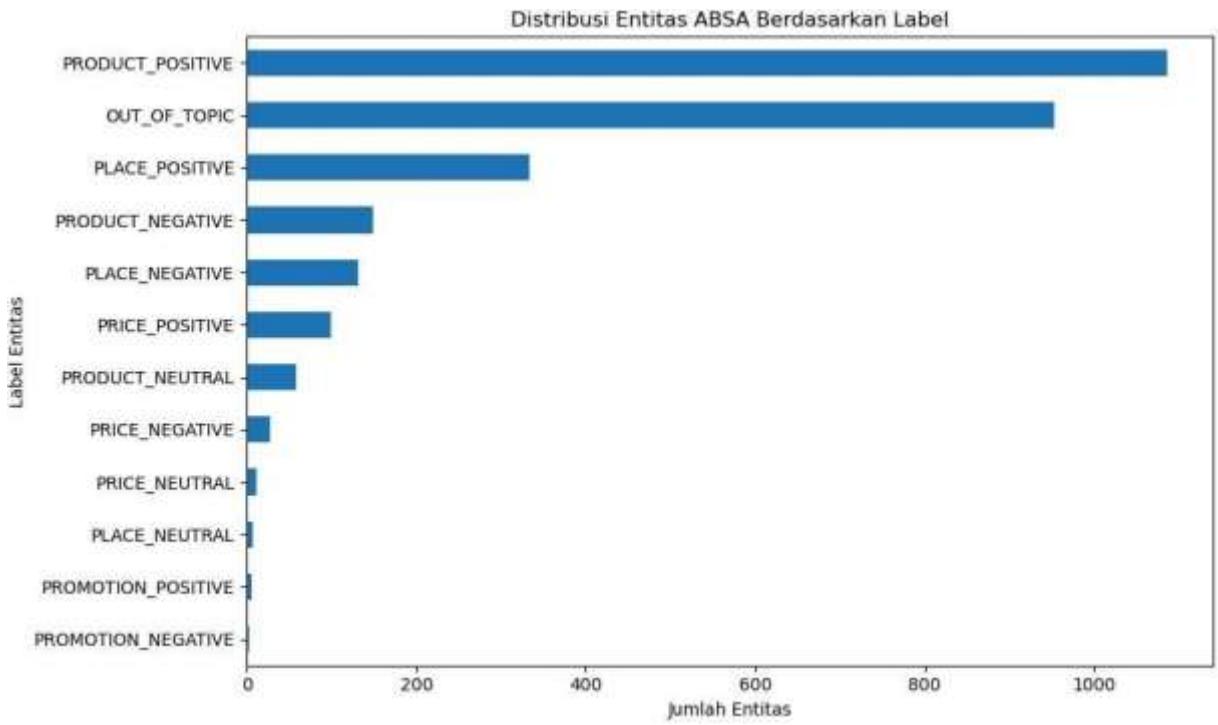
Out[20]:

```
entity_label
PRODUCT_POSITIVE      1086
OUT_OF_TOPIC           953
PLACE_POSITIVE         334
PRODUCT_NEGATIVE       149
PLACE_NEGATIVE         131
PRICE_POSITIVE          99
PRODUCT_NEUTRAL         58
PRICE_NEGATIVE          27
PRICE_NEUTRAL           11
PLACE_NEUTRAL            7
PROMOTION_POSITIVE       5
PROMOTION_NEGATIVE       2
Name: count, dtype: int64
```

In [21]:

```
# Visualisasi Distribusi Entitas ABSA

plt.figure(figsize=(10, 6))
entity_label_counts.sort_values().plot(kind="barh")
plt.title("Distribusi Entitas ABSA Berdasarkan Label")
plt.xlabel("Jumlah Entitas")
plt.ylabel("Label Entitas")
```

```
plt.tight_layout()
plt.show()
```



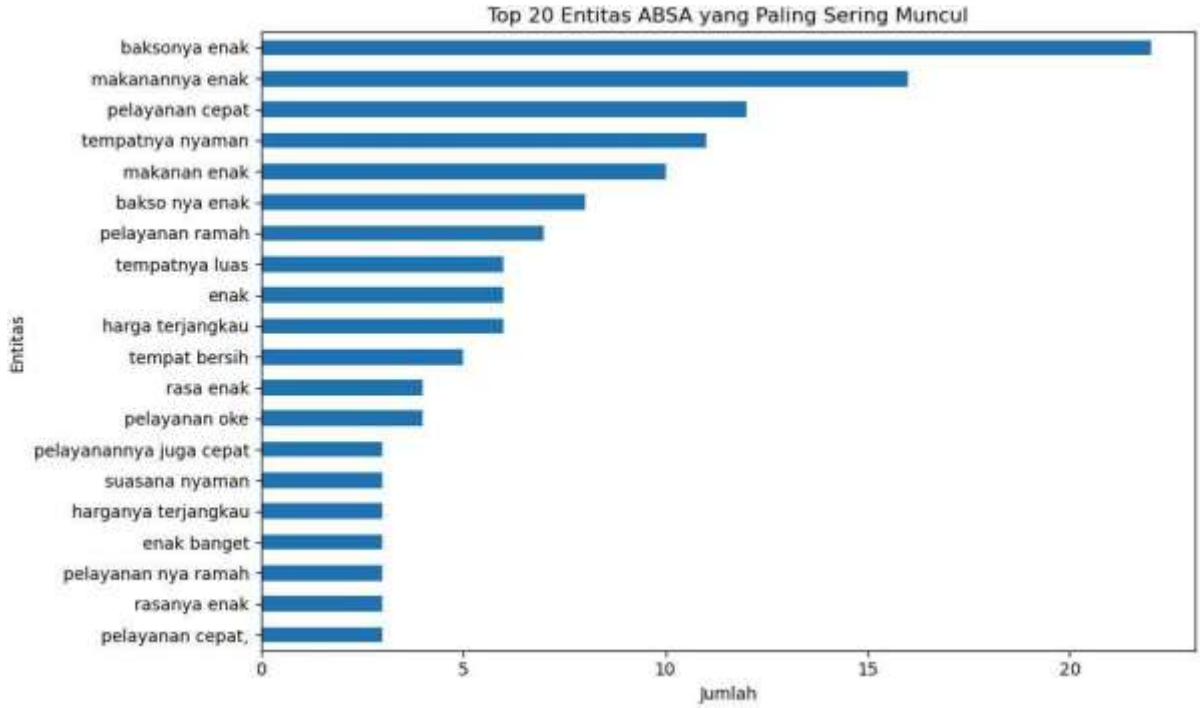
In [22]:

```
# ========================
# 8. Top 20 Entitas yang Paling Sering Muncul
# ========================

top_entities = entity_df["entity_text"].str.lower().value_counts().head(20)

print("Top 20 Entitas ABSA:")
top_entities
```

```
Top 20 Entitas ABSA:
```

Out[22]:
```
entity_text
baksonya enak              22
makanannya enak            16
pelayanan cepat            12
tempatnya nyaman           11
makanan enak               10
bakso nya enak              8
pelayanan ramah             7
enak                        6
tempatnya luas              6
harga terjangkau            6
tempat bersih               5
rasa enak                   4
pelayanan oke               4
suasana nyaman              3
harganya terjangkau         3
enak banget                 3
pelayanan nya ramah         3
rasanya enak                3
pelayanannya juga cepat     3
pelayanan cepat,            3
Name: count, dtype: int64
```

In [23]:
```
# Visualisasi Top 20 Entitas

plt.figure(figsize=(10, 6))
top_entities.sort_values().plot(kind="barh")
plt.title("Top 20 Entitas ABSA yang Paling Sering Muncul")
plt.xlabel("Jumlah")
plt.ylabel("Entitas")
plt.tight_layout()
plt.show()
```



Distribusi Panjang Review

In [24]:
```
# ========================
# 9. Hitung Panjang Review
# ========================

df_eda["word_count"] = df_eda["text"].apply(lambda x: len(str(x).split()))
df_eda["char_count"] = df_eda["text"].apply(lambda x: len(str(x)))

df_eda[["text", "word_count", "char_count"]].head()
```

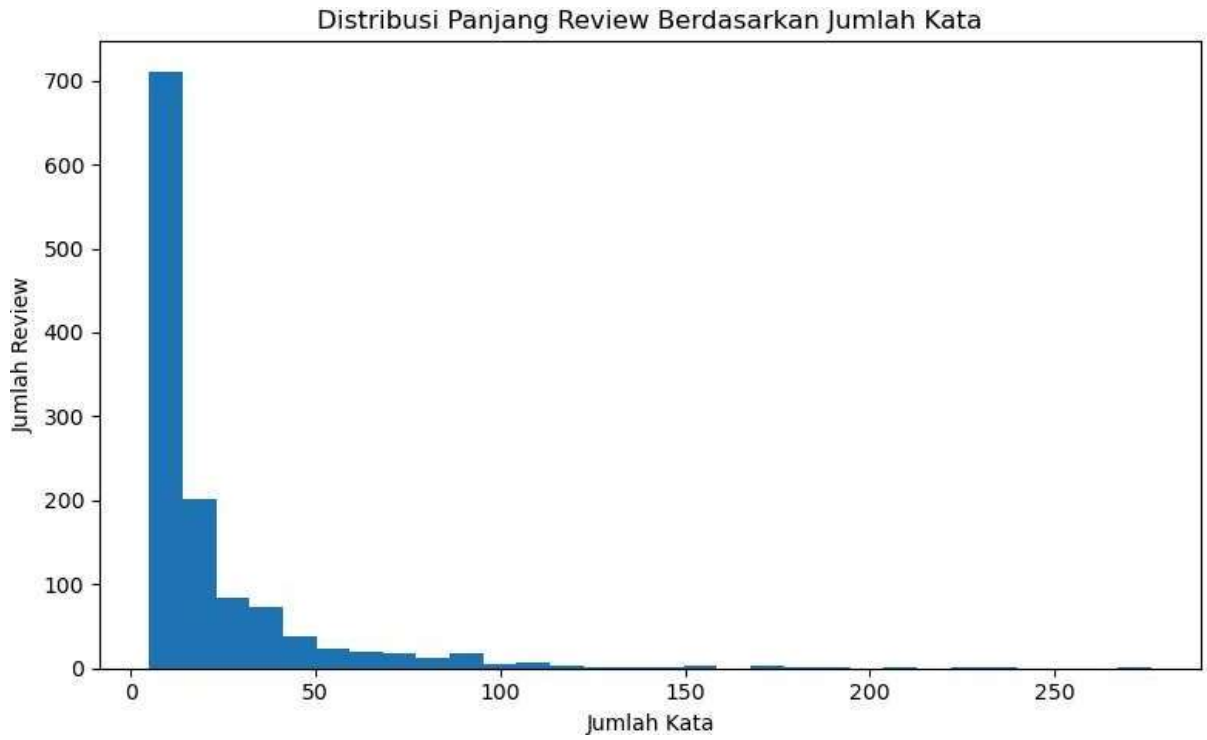
Out[24]:

	text	word_count	char_count
0	Semuanya worth bakso mozanya enak banget	6	40
1	Bakso bakar ya enak dan pas	6	27
2	Saya kesini jam 3 sore bersama keluarga, yg sa...	111	721
3	Ud 3x ksni tiap k jogja, baksonya enak, saran ...	33	166
4	servisnya oke banget. baru dateng, mba pelayan...	14	94

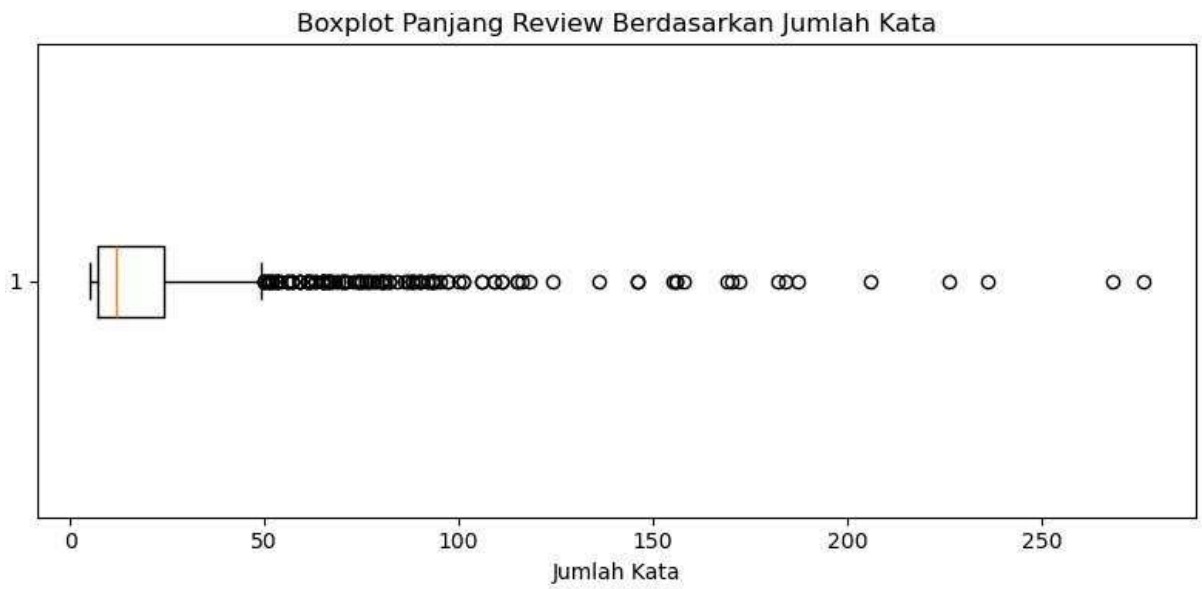
In [25]:
```
# Statistik Panjang Review Berdasarkan Jumlah Kata

df_eda["word_count"].describe()
```

Out[25]:
```
count    1232.000000
mean       22.609578
std        29.033934
min         5.000000
25%         7.000000
50%        12.000000
75%        24.000000
max       276.000000
Name: word_count, dtype: float64
```

In [26]:
```
# Visualisasi Histogram Panjang Review

plt.figure(figsize=(8, 5))
plt.hist(df_eda["word_count"], bins=30)
plt.title("Distribusi Panjang Review Berdasarkan Jumlah Kata")
plt.xlabel("Jumlah Kata")
plt.ylabel("Jumlah Review")
plt.tight_layout()
plt.show()
```



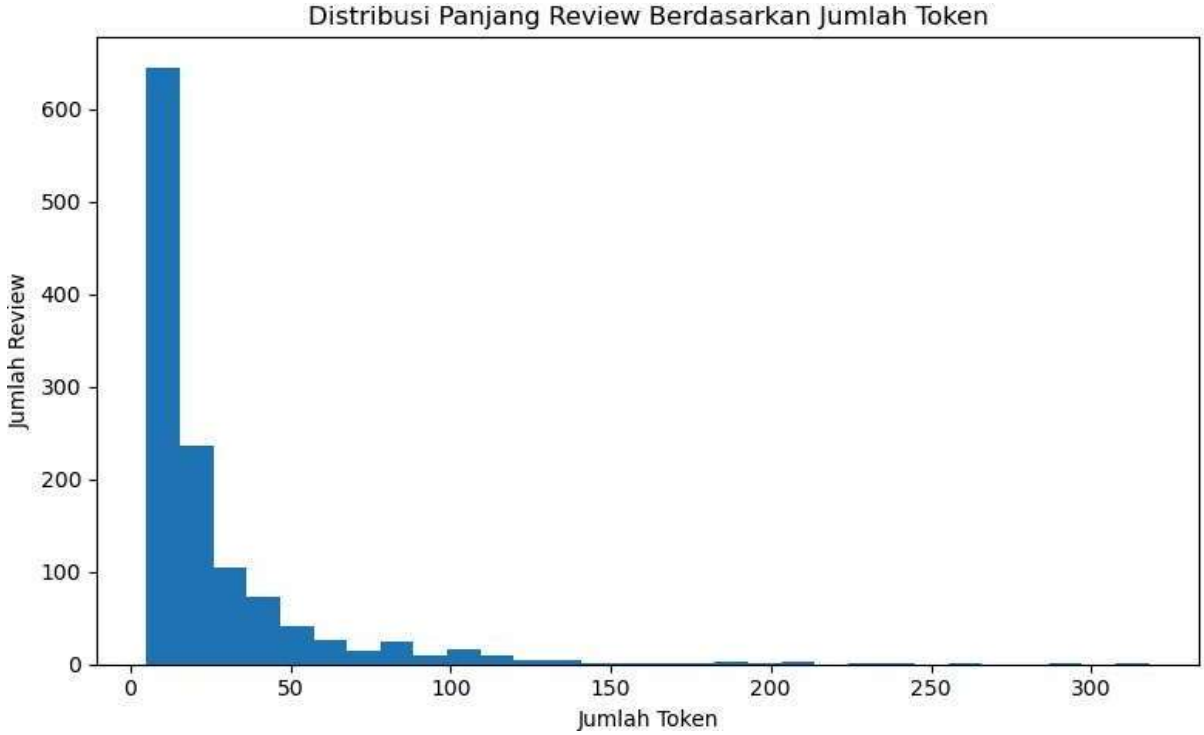
In [27]:
```
# Visualisasi Boxplot Panjang Review

plt.figure(figsize=(8, 4))
plt.boxplot(df_eda["word_count"], vert=False)
plt.title("Boxplot Panjang Review Berdasarkan Jumlah Kata")
plt.xlabel("Jumlah Kata")
plt.tight_layout()
plt.show()
```



In [28]:
```
# Distribusi Panjang Review Berdasarkan Token jika kolom tokens tersedia

if "tokens" in df_eda.columns:
    df_eda["token_count"] = df_eda["tokens"].apply(lambda x: len(x) if isinstance(x
```

```
    print("Statistik jumlah token:")
    print(df_eda["token_count"].describe())

    plt.figure(figsize=(8, 5))
    plt.hist(df_eda["token_count"], bins=30)
    plt.title("Distribusi Panjang Review Berdasarkan Jumlah Token")
    plt.xlabel("Jumlah Token")
    plt.ylabel("Jumlah Review")
    plt.tight_layout()
    plt.show()
else:
    print("Kolom tokens tidak tersedia, analisis token dilewati.")
```

```
Statistik jumlah token:
count    1232.000000
mean       26.637987
std        34.217094
min         5.000000
25%         9.000000
50%        15.000000
75%        29.000000
max       318.000000
Name: token_count, dtype: float64
```



Matriks Korelasi Antar Label

In [29]:

```
# ========================
# 10. Ubah Multi-label menjadi Format Biner
# ========================

mlb = MultiLabelBinarizer()

label_matrix = mlb.fit_transform(df_eda["accept"])
label_df = pd.DataFrame(label_matrix, columns=mlb.classes_)
```

```
label_df.head()
```

Out[29]:

	OUT_OF_TOPIC	PLACE_NEGATIVE	PLACE_NEUTRAL	PLACE_POSITIVE	PRICE_NEGATIVE
0	0	0	0	0	0
1	0	0	0	0	0
2	1	1	0	0	0
3	1	1	0	0	0
4	1	0	0	0	0

In [30]:

```
# ========================
# 11. Hitung Korelasi Antar Label
# ========================

corr_matrix = label_df.corr()

corr_matrix
```

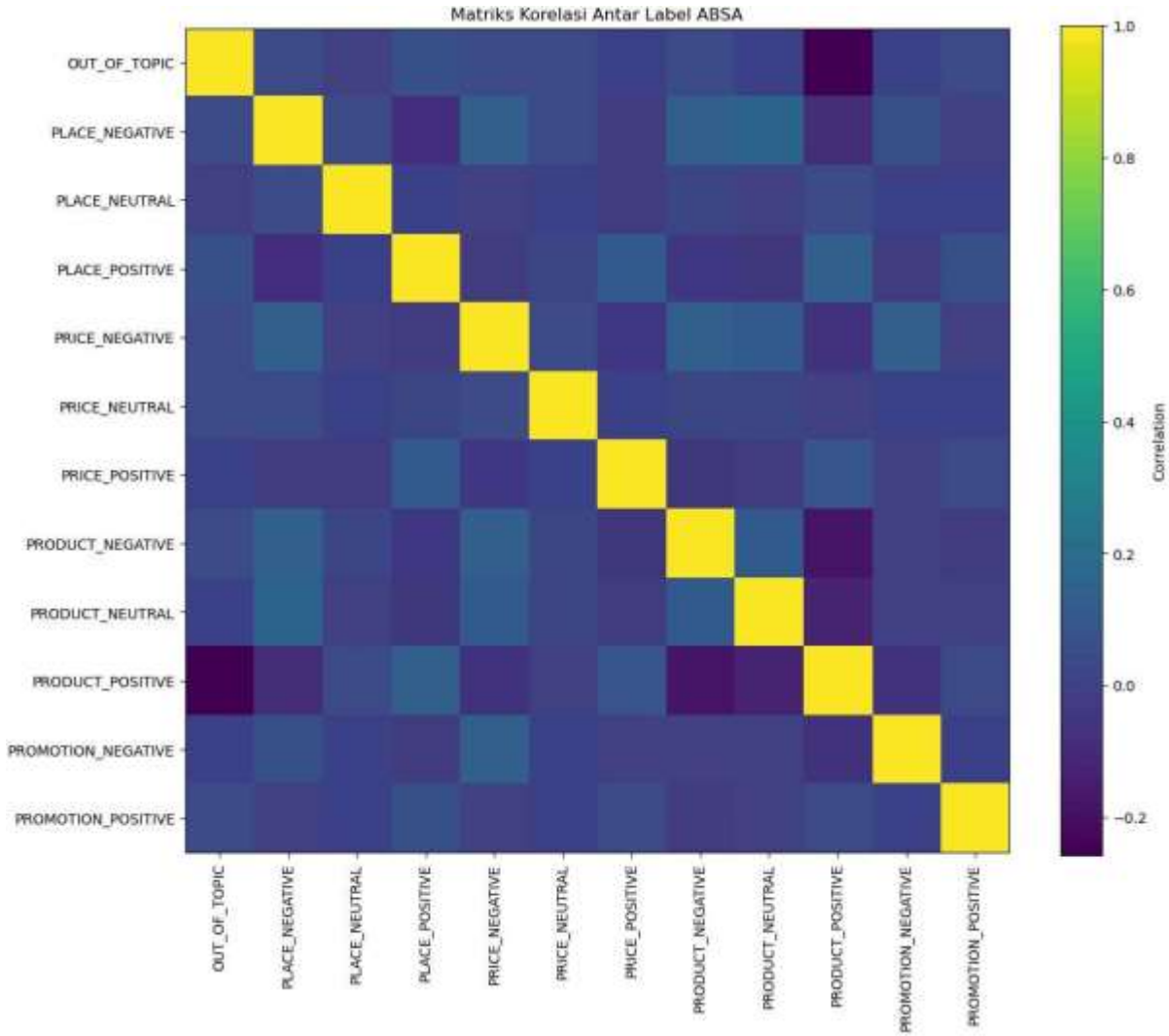
Out[30]:

	OUT_OF_TOPIC	PLACE_NEGATIVE	PLACE_NEUTRAL	PLACE_POSITIV
OUT_OF_TOPIC	1.000000	0.036206	-0.012042	0.07075
PLACE_NEGATIVE	0.036206	1.000000	0.028095	-0.07727
PLACE_NEUTRAL	-0.012042	0.028095	1.000000	-0.00647
PLACE_POSITIVE	0.070752	-0.077270	-0.006471	1.00000
PRICE_NEGATIVE	0.031554	0.135707	-0.008805	-0.02236
PRICE_NEUTRAL	0.040895	0.027917	-0.006592	0.01544
PRICE_POSITIVE	-0.005599	-0.021477	-0.018870	0.10360
PRODUCT_NEGATIVE	0.045501	0.130953	0.023404	-0.04552
PRODUCT_NEUTRAL	0.002386	0.145851	-0.013401	-0.05325
PRODUCT_POSITIVE	-0.257762	-0.091366	0.036855	0.12228
PROMOTION_NEGATIVE	0.000458	0.062247	-0.002574	-0.02287
PROMOTION_POSITIVE	0.029187	-0.016871	-0.003643	0.06735

In [31]:

```
# Visualisasi Matriks Korelasi Antar Label

plt.figure(figsize=(12, 10))
plt.imshow(corr_matrix)
plt.colorbar(label="Correlation")

plt.xticks(
```

```
    ticks=range(len(corr_matrix.columns)),
    labels=corr_matrix.columns,
    rotation=90
)

plt.yticks(
    ticks=range(len(corr_matrix.columns)),
    labels=corr_matrix.columns
)

plt.title("Matriks Korelasi Antar Label ABSA")
plt.tight_layout()
plt.show()
```



In [32]:
```
# ========================
# 12. Ringkasan Statistik Dataset
# ========================

summary = {
    "jumlah_review_eda": len(df_eda),
    "jumlah_label_unik": len(label_counts),
    "jumlah_entitas": len(entity_df),
    "rata_rata_panjang_kata": df_eda["word_count"].mean(),
```

```
    "median_panjang_kata": df_eda["word_count"].median(),
    "min_panjang_kata": df_eda["word_count"].min(),
    "max_panjang_kata": df_eda["word_count"].max()
}

summary_df = pd.DataFrame([summary])

summary_df
```

Out[32]:

	jumlah_review_eda	jumlah_label_unik	jumlah_entitas	rata_rata_panjang_kata	median_pa
0	1232	12	2862	22.609578	

STREAMLIT

```
import os
import json
import re
import itertools
from collections import defaultdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import streamlit as st
from sklearn.metrics import cohen_kappa_score


# ==========================
# KONFIGURASI FILE
# ==========================

CLEAN_CSV = "Kelp4_dataset_2.csv"
ANNOTATION_JSONL = "Kelp4_dataset_anotasi.jsonl"

LABELS = [
    "PRODUCT_POSITIVE", "PRODUCT_NEGATIVE", "PRODUCT_NEUTRAL",
    "PRICE_POSITIVE", "PRICE_NEGATIVE", "PRICE_NEUTRAL",
    "PLACE_POSITIVE", "PLACE_NEGATIVE", "PLACE_NEUTRAL",
    "PROMOTION_POSITIVE", "PROMOTION_NEGATIVE",
"PROMOTION_NEUTRAL",
    "OUT_OF_TOPIC"
]

ASPECTS = [
    "PRODUCT",
    "PRICE",
    "PLACE",
    "PROMOTION",
```

```
    "OUT_OF_TOPIC"
]


# ==========================
# KONFIGURASI STREAMLIT
# ==========================

st.set_page_config(
    page_title="Kelp4 Dataset Explorer",
    page_icon="📊",
    layout="wide"
)


# ==========================
# FUNGSI BANTUAN UMUM
# ==========================

def normalize_text(text):
    if pd.isna(text):
        return ""
    text = str(text).lower()
    text = re.sub(r"\s+", " ", text)
    text = text.strip()
    return text


def get_aspect(label):
    if label == "OUT_OF_TOPIC":
        return "OUT_OF_TOPIC"
    return label.split("_")[0]


def get_sentiment(label):
    if label == "OUT_OF_TOPIC":
        return "OUT_OF_TOPIC"
```

```
    return label.split("_")[-1]


def has_aspect(labels, aspect):
    if not isinstance(labels, (list, set, tuple)):
        return False

    if aspect == "OUT_OF_TOPIC":
        return "OUT_OF_TOPIC" in labels

    return any(str(label).startswith(aspect + "_") for label in labels)


def safe_list(value):
    if isinstance(value, list):
        return value
    return []


def safe_kappa(y1, y2):
    """
    Cohen's Kappa tidak bermakna jika semua nilai hanya satu kelas.
    Jika semua nilai sama dan tidak ada variasi, fungsi mengembalikan NaN.
    """
    if len(y1) == 0 or len(y2) == 0:
        return np.nan

    unique_values = set(y1 + y2)

    if len(unique_values) < 2:
        return np.nan

    return cohen_kappa_score(y1, y2)


def interpret_kappa(value):
    if pd.isna(value):
```

```
        return "Tidak dapat dihitung"

    if value < 0:
        return "Sangat rendah / tidak ada kesepakatan"
    elif value <= 0.20:
        return "Sangat rendah"
    elif value <= 0.40:
        return "Rendah"
    elif value <= 0.60:
        return "Sedang"
    elif value <= 0.80:
        return "Baik"
    else:
        return "Sangat baik"


def read_csv_auto(path):
    if not os.path.exists(path):
        st.error(f"File {path} tidak ditemukan.")
        st.stop()

    candidates = []

    for sep in [";", ","]:
        try:
            df = pd.read_csv(path, sep=sep, encoding="utf-8-sig")
            candidates.append(df)
        except Exception:
            pass

    if len(candidates) == 0:
        st.error(f"File {path} gagal dibaca.")
        st.stop()

    df = max(candidates, key=lambda x: len(x.columns))
    return df
```

```
def count_table(series, index_name, value_name="jumlah"):
    return series.rename(value_name).rename_axis(index_name).reset_index()


# ==========================
# LOAD DATASET
# ==========================

@st.cache_data
def load_clean_dataset(path):
    df = read_csv_auto(path)

    if "review_text" not in df.columns:
        if "text" in df.columns:
            df["review_text"] = df["text"]
        else:
            st.error("Kolom review_text atau text tidak ditemukan di dataset CSV.")
            st.stop()

    if "category" not in df.columns:
        df["category"] = "Tidak diketahui"

    if "business_name" not in df.columns:
        df["business_name"] = "Tidak diketahui"

    if "rating" not in df.columns:
        df["rating"] = np.nan

    df["review_text"] = df["review_text"].fillna("").astype(str)
    df["word_count"] = df["review_text"].apply(lambda x: len(str(x).split()))
    df["text_key"] = df["review_text"].apply(normalize_text)

    return df


@st.cache_data
```

```
def load_annotation_dataset(path):
    if not os.path.exists(path):
        st.error(f"File {path} tidak ditemukan.")
        st.stop()

    data = []

    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                data.append(json.loads(line))

    df_raw = pd.DataFrame(data)

    if "text" not in df_raw.columns:
        st.error("Kolom text tidak ditemukan di file anotasi JSONL.")
        st.stop()

    if "accept" not in df_raw.columns:
        df_raw["accept"] = [[] for _ in range(len(df_raw))]

    if "spans" not in df_raw.columns:
        df_raw["spans"] = [[] for _ in range(len(df_raw))]

    df_raw["accept"] = df_raw["accept"].apply(safe_list)
    df_raw["spans"] = df_raw["spans"].apply(safe_list)
    df_raw["text"] = df_raw["text"].fillna("").astype(str)
    df_raw["text_key"] = df_raw["text"].apply(normalize_text)
    df_raw["word_count"] = df_raw["text"].apply(lambda x: len(str(x).split()))

    # Untuk EDA dan visualisasi, ambil satu annotator agar review tidak terhitung ganda.
    df_single = df_raw.copy()

    if "_annotator_id" in df_single.columns and "_input_hash" in df_single.columns:
        main_annotator = df_single["_annotator_id"].value_counts().idxmax()
        df_single = df_single[df_single["_annotator_id"] == main_annotator].copy()
```

```
        df_single = df_single.drop_duplicates(subset=["_input_hash"])
    else:
        main_annotator = "Tidak tersedia"

    df_single["labels_str"] = df_single["accept"].apply(
        lambda labels: ", ".join(labels) if labels else "-"
    )

    df_single["aspects"] = df_single["accept"].apply(
        lambda labels: sorted(set(get_aspect(label) for label in labels))
    )

    df_single["sentiments"] = df_single["accept"].apply(
        lambda labels: sorted(set(get_sentiment(label) for label in labels))
    )

    df_single["aspects_str"] = df_single["aspects"].apply(
        lambda x: ", ".join(x) if x else "-"
    )

    df_single["sentiments_str"] = df_single["sentiments"].apply(
        lambda x: ", ".join(x) if x else "-"
    )

    return df_raw, df_single, main_annotator


# ==========================
# FUNGSI ENTITAS ABSA / NER
# ==========================

def build_entity_dataframe(df_absa):
    entities = []

    for _, row in df_absa.iterrows():
        text = row.get("text", "")
        spans = row.get("spans", [])
```

```
        if not isinstance(spans, list):
            continue

        for span in spans:
            start = span.get("start")
            end = span.get("end")
            label = span.get("label")

            if start is not None and end is not None:
                entity_text = text[start:end]

                entities.append({
                    "review_text": text,
                    "entity_text": entity_text,
                    "entity_label": label,
                    "start": start,
                    "end": end
                })

    if len(entities) == 0:
        return pd.DataFrame(
            columns=["review_text", "entity_text", "entity_label", "start", "end"]
        )

    return pd.DataFrame(entities)


def add_entity_string(df_absa):
    entity_strings = []

    for _, row in df_absa.iterrows():
        text = row.get("text", "")
        spans = row.get("spans", [])

        ent_list = []
```

```
        if isinstance(spans, list):
            for span in spans:
                start = span.get("start")
                end = span.get("end")
                label = span.get("label")

                if start is not None and end is not None:
                    entity_text = text[start:end]
                    ent_list.append(f"{entity_text} ({label})")

        entity_strings.append("; ".join(ent_list) if ent_list else "-")

    df_absa["entities_str"] = entity_strings
    return df_absa


# ==========================
# FUNGSI IRR / COHEN'S KAPPA
# ==========================

def build_annotation_records(df):
    records = defaultdict(dict)
    text_lookup = {}

    if "_input_hash" not in df.columns or "_annotator_id" not in df.columns:
        return records, text_lookup

    for _, row in df.iterrows():
        input_hash = row.get("_input_hash")
        annotator = row.get("_annotator_id")
        labels = set(safe_list(row.get("accept", [])))
        text = row.get("text", "")

        if pd.isna(input_hash) or pd.isna(annotator):
            continue

        records[input_hash][annotator] = labels
```

```
        text_lookup[input_hash] = text

    return records, text_lookup


def compute_kappa_by_label(df, labels):
    records, _ = build_annotation_records(df)

    if len(records) == 0:
        return pd.DataFrame()

    annotators = df["_annotator_id"].dropna().value_counts().index.tolist()

    if len(annotators) < 2:
        return pd.DataFrame()

    pairs = list(itertools.combinations(annotators, 2))
    results = []

    for ann1, ann2 in pairs:
        paired_items = {
            key: value
            for key, value in records.items()
            if ann1 in value and ann2 in value
        }

        for label in labels:
            y1 = []
            y2 = []

            for item in paired_items.values():
                y1.append(1 if label in item[ann1] else 0)
                y2.append(1 if label in item[ann2] else 0)

            if len(y1) == 0:
                continue
```

```
        kappa = safe_kappa(y1, y2)
        agreement = np.mean([a == b for a, b in zip(y1, y2)]) * 100

        results.append({
            "pair": f"{ann1} vs {ann2}",
            "label": label,
            "cohen_kappa": kappa,
            "interpretasi": interpret_kappa(kappa),
            "percent_agreement": agreement,
            "jumlah_data_berpasangan": len(y1),
            "annotator_1_yes": int(sum(y1)),
            "annotator_2_yes": int(sum(y2)),
            "both_yes": int(sum(a == 1 and b == 1 for a, b in zip(y1, y2)))
        })

    pair_df = pd.DataFrame(results)

    if len(pair_df) == 0:
        return pair_df

    # Jika hanya ada 2 annotator, hasilnya sama.
    # Jika lebih dari 2, rata-rata Kappa per label dihitung dari semua pasangan annotator.
    label_df = (
        pair_df
        .groupby("label", as_index=False)
        .agg({
            "cohen_kappa": "mean",
            "percent_agreement": "mean",
            "jumlah_data_berpasangan": "max",
            "annotator_1_yes": "sum",
            "annotator_2_yes": "sum",
            "both_yes": "sum"
        })
    )

    label_df["interpretasi"] = label_df["cohen_kappa"].apply(interpret_kappa)
    label_df = label_df.sort_values("cohen_kappa", ascending=False, na_position="last")
```

```
    return label_df


def compute_kappa_by_aspect(df, aspects):
    records, _ = build_annotation_records(df)

    if len(records) == 0:
        return pd.DataFrame()

    annotators = df["_annotator_id"].dropna().value_counts().index.tolist()

    if len(annotators) < 2:
        return pd.DataFrame()

    pairs = list(itertools.combinations(annotators, 2))
    results = []

    for ann1, ann2 in pairs:
        paired_items = {
            key: value
            for key, value in records.items()
            if ann1 in value and ann2 in value
        }

        for aspect in aspects:
            y1 = []
            y2 = []

            for item in paired_items.values():
                y1.append(1 if has_aspect(item[ann1], aspect) else 0)
                y2.append(1 if has_aspect(item[ann2], aspect) else 0)

            if len(y1) == 0:
                continue

            kappa = safe_kappa(y1, y2)
```

```
        agreement = np.mean([a == b for a, b in zip(y1, y2)]) * 100

        results.append({
            "pair": f"{ann1} vs {ann2}",
            "aspek": aspect,
            "cohen_kappa": kappa,
            "interpretasi": interpret_kappa(kappa),
            "percent_agreement": agreement,
            "jumlah_data_berpasangan": len(y1),
            "annotator_1_yes": int(sum(y1)),
            "annotator_2_yes": int(sum(y2)),
            "both_yes": int(sum(a == 1 and b == 1 for a, b in zip(y1, y2)))
        })

    pair_df = pd.DataFrame(results)

    if len(pair_df) == 0:
        return pair_df

    aspect_df = (
        pair_df
        .groupby("aspek", as_index=False)
        .agg({
            "cohen_kappa": "mean",
            "percent_agreement": "mean",
            "jumlah_data_berpasangan": "max",
            "annotator_1_yes": "sum",
            "annotator_2_yes": "sum",
            "both_yes": "sum"
        })
    )

    aspect_df["interpretasi"] = aspect_df["cohen_kappa"].apply(interpret_kappa)
    aspect_df = aspect_df.sort_values("cohen_kappa", ascending=False, na_position="last")

    return aspect_df
```

```
def get_paired_review_count(df):
    if "_input_hash" not in df.columns or "_annotator_id" not in df.columns:
        return 0

    counts = df.groupby("_input_hash")["_annotator_id"].nunique()
    return int((counts >= 2).sum())


def get_disagreement_examples(df, max_examples=50):
    records, text_lookup = build_annotation_records(df)

    if len(records) == 0:
        return pd.DataFrame()

    annotators = df["_annotator_id"].dropna().value_counts().index.tolist()

    if len(annotators) < 2:
        return pd.DataFrame()

    ann1, ann2 = annotators[0], annotators[1]

    examples = []

    for input_hash, item in records.items():
        if ann1 not in item or ann2 not in item:
            continue

        labels_1 = item[ann1]
        labels_2 = item[ann2]

        if labels_1 != labels_2:
            examples.append({
                "review_text": text_lookup.get(input_hash, ""),
                f"label_{ann1}": ", ".join(sorted(labels_1)) if labels_1 else "-",
                f"label_{ann2}": ", ".join(sorted(labels_2)) if labels_2 else "-",
                "label_sama": ", ".join(sorted(labels_1.intersection(labels_2))) if
labels_1.intersection(labels_2) else "-",
```

```
            "hanya_annotator_1": ", ".join(sorted(labels_1 - labels_2)) if labels_1 - labels_2
else "-",
            "hanya_annotator_2": ", ".join(sorted(labels_2 - labels_1)) if labels_2 - labels_1
else "-"
        })

    if len(examples) >= max_examples:
        break

    return pd.DataFrame(examples)


# ==========================
# FUNGSI FILTER
# ==========================

def filter_by_label_aspect_sentiment(df, selected_aspect, selected_sentiment,
selected_label):
    filtered = df.copy()

    def match_filter(labels):
        if not isinstance(labels, list):
            return False

        if selected_label != "Semua":
            return selected_label in labels

        if selected_aspect == "Semua" and selected_sentiment == "Semua":
            return True

        for label in labels:
            aspect = get_aspect(label)
            sentiment = get_sentiment(label)

            aspect_ok = selected_aspect == "Semua" or aspect == selected_aspect
            sentiment_ok = selected_sentiment == "Semua" or sentiment == selected_sentiment

            if aspect_ok and sentiment_ok:
```

```
            return True

        return False

    filtered = filtered[filtered["accept"].apply(match_filter)]
    return filtered


# ==========================
# FUNGSI PLOT
# ==========================

def plot_bar(series, title, xlabel, ylabel):
    if len(series) == 0:
        st.warning("Tidak ada data untuk divisualisasikan.")
        return

    fig, ax = plt.subplots(figsize=(10, 5))
    series.sort_values().plot(kind="barh", ax=ax)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    plt.tight_layout()
    st.pyplot(fig)


def plot_hist(data, title, xlabel, ylabel):
    if len(data) == 0:
        st.warning("Tidak ada data untuk divisualisasikan.")
        return

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.hist(data, bins=30)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    plt.tight_layout()
```

```
    st.pyplot(fig)


def plot_kappa_bar(df, label_col, value_col, title):
    if len(df) == 0:
        st.warning("Tidak ada data Kappa untuk divisualisasikan.")
        return

    plot_data = df.dropna(subset=[value_col]).set_index(label_col)[value_col]

    if len(plot_data) == 0:
        st.warning("Nilai Kappa kosong atau tidak dapat dihitung.")
        return

    fig, ax = plt.subplots(figsize=(10, 5))
    plot_data.sort_values().plot(kind="barh", ax=ax)
    ax.set_title(title)
    ax.set_xlabel("Cohen's Kappa")
    ax.set_ylabel(label_col)
    plt.tight_layout()
    st.pyplot(fig)


# ==========================
# LOAD DATA
# ==========================

df_clean = load_clean_dataset(CLEAN_CSV)
df_annot_raw, df_absa, main_annotator =
load_annotation_dataset(ANNOTATION_JSONL)
df_absa = add_entity_string(df_absa)
entity_df = build_entity_dataframe(df_absa)

# Gabungkan metadata CSV ke data anotasi berdasarkan teks review
meta_cols = ["text_key", "category", "business_name", "rating"]
df_meta = df_clean[meta_cols].drop_duplicates(subset=["text_key"])

df_absa_full = df_absa.merge(df_meta, on="text_key", how="left")
```

```
df_label_meta = df_absa[["text_key", "accept", "labels_str",
"entities_str"]].drop_duplicates(subset=["text_key"])
df_browse = df_clean.merge(df_label_meta, on="text_key", how="left")

df_browse["labels_str"] = df_browse["labels_str"].fillna("-")
df_browse["entities_str"] = df_browse["entities_str"].fillna("-")
df_browse["accept"] = df_browse["accept"].apply(safe_list)


# ==========================
# HITUNG IRR / KAPPA
# ==========================

kappa_label_df = compute_kappa_by_label(df_annot_raw, LABELS)
kappa_aspect_df = compute_kappa_by_aspect(df_annot_raw, ASPECTS)
disagreement_df = get_disagreement_examples(df_annot_raw, max_examples=100)


# ==========================
# SIDEBAR
# ==========================

st.sidebar.title("Kelp4 Dataset Explorer")

page = st.sidebar.radio(
    "Pilih Halaman",
    [
        "Overview",
        "Browse Reviews",
        "Panel ABSA",
        "Statistics",
        "IRR Report"
    ]
)

st.sidebar.markdown("---")
st.sidebar.subheader("Filter Umum")
```

```
category_options = ["Semua"] +
sorted(df_clean["category"].dropna().astype(str).unique().tolist())
selected_category = st.sidebar.selectbox("Filter kategori usaha", category_options)

aspect_options = ["Semua", "PRODUCT", "PRICE", "PLACE", "PROMOTION",
"OUT_OF_TOPIC"]
selected_aspect = st.sidebar.selectbox("Filter aspek", aspect_options)

sentiment_options = ["Semua", "POSITIVE", "NEGATIVE", "NEUTRAL",
"OUT_OF_TOPIC"]
selected_sentiment = st.sidebar.selectbox("Filter sentimen", sentiment_options)

label_options = ["Semua"] + LABELS
selected_label = st.sidebar.selectbox("Filter label aspek-sentimen", label_options)

search_query = st.sidebar.text_input("Cari kata dalam review")



# ==========================
# TERAPKAN FILTER
# ==========================

filtered_browse = df_browse.copy()

if selected_category != "Semua":
    filtered_browse = filtered_browse[filtered_browse["category"].astype(str) ==
selected_category]

if search_query.strip() != "":
    q = search_query.lower().strip()
    filtered_browse = filtered_browse[
        filtered_browse["review_text"].astype(str).str.lower().str.contains(q, na=False)
    ]

filtered_browse = filter_by_label_aspect_sentiment(
    filtered_browse,
    selected_aspect,
```

```
    selected_sentiment,
    selected_label
)

filtered_absa = df_absa_full.copy()

if selected_category != "Semua" and "category" in filtered_absa.columns:
    filtered_absa = filtered_absa[filtered_absa["category"].astype(str) == selected_category]

if search_query.strip() != "":
    q = search_query.lower().strip()
    filtered_absa = filtered_absa[
        filtered_absa["text"].astype(str).str.lower().str.contains(q, na=False)
    ]

filtered_absa = filter_by_label_aspect_sentiment(
    filtered_absa,
    selected_aspect,
    selected_sentiment,
    selected_label
)


# ==========================
# HALAMAN OVERVIEW
# ==========================

if page == "Overview":
    st.title("📊 Kelp4 Dataset Explorer")
    st.write("Aplikasi ini digunakan untuk mengeksplorasi dataset review Google Places dan hasil anotasi ABSA.")

    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric("Jumlah Review Bersih", len(df_clean))
```

```
    with col2:
        st.metric("Jumlah Review Anotasi", len(df_absa))

    with col3:
        st.metric("Jumlah Entitas ABSA", len(entity_df))

    with col4:
        total_labels = df_absa["accept"].explode().nunique()
        st.metric("Jumlah Label Unik", total_labels)

    st.markdown("### Informasi Dataset")
    st.write(f"File dataset bersih: `{CLEAN_CSV}`")
    st.write(f"File dataset anotasi: `{ANNOTATION_JSONL}`")
    st.write(f"Annotator yang digunakan untuk visualisasi EDA: `{main_annotator}`")

    st.markdown("### Distribusi Kategori Usaha")
    category_counts = df_clean["category"].value_counts()
    st.dataframe(count_table(category_counts, "category"), use_container_width=True)
    plot_bar(category_counts, "Distribusi Kategori Usaha", "Jumlah Review", "Kategori")

    if "rating" in df_clean.columns:
        st.markdown("### Distribusi Rating")
        rating_counts = df_clean["rating"].value_counts().sort_index()
        st.bar_chart(rating_counts)

    st.markdown("### Contoh Data Review")
    preview_cols = ["category", "business_name", "review_text", "rating"]
    preview_cols = [col for col in preview_cols if col in df_clean.columns]
    st.dataframe(df_clean[preview_cols].head(20), use_container_width=True)


# ==========================
# HALAMAN BROWSE REVIEWS
# ==========================

elif page == "Browse Reviews":
    st.title("🔎 Browse Reviews")
```

```
    st.write("Halaman ini menampilkan daftar review dengan informasi kategori, nama usaha, teks review, rating, label ABSA, dan entitas.")

    st.info(f"Jumlah data setelah filter: {len(filtered_browse)}")

    browse_cols = [
        "category",
        "business_name",
        "review_text",
        "rating",
        "labels_str",
        "entities_str",
        "word_count"
    ]

    existing_cols = [col for col in browse_cols if col in filtered_browse.columns]

    st.dataframe(
        filtered_browse[existing_cols],
        use_container_width=True,
        height=600
    )


# ==========================
# HALAMAN PANEL ABSA
# ==========================

elif page == "Panel ABSA":
    st.title("🏷️ Panel ABSA")
    st.write("Halaman ini menampilkan hasil anotasi ABSA dalam bentuk label multi-label dan entitas NER.")

    st.info(f"Jumlah data anotasi setelah filter: {len(filtered_absa)}")

    st.markdown("### Dataset Tabular Label ABSA")

    absa_cols = [
```

```
        "category",
        "business_name",
        "rating",
        "text",
        "labels_str",
        "aspects_str",
        "sentiments_str",
        "entities_str"
    ]

    existing_absa_cols = [col for col in absa_cols if col in filtered_absa.columns]

    st.dataframe(
        filtered_absa[existing_absa_cols],
        use_container_width=True,
        height=500
    )

    st.markdown("### Dataset Tabular Entitas NER")

    filtered_text_keys = set(filtered_absa["text_key"].tolist())

    entity_df_display = entity_df.copy()
    entity_df_display["text_key"] = entity_df_display["review_text"].apply(normalize_text)
    entity_df_display =
entity_df_display[entity_df_display["text_key"].isin(filtered_text_keys)]

    if len(entity_df_display) > 0:
        st.dataframe(
            entity_df_display[["entity_text", "entity_label", "start", "end", "review_text"]],
            use_container_width=True,
            height=500
        )
    else:
        st.warning("Tidak ada entitas NER yang sesuai dengan filter.")

# ==========================
```

```
# HALAMAN STATISTICS
# ==========================

elif page == "Statistics":
    st.title("📈 Statistics")
    st.write("Halaman ini menampilkan ringkasan statistik dataset, distribusi panjang review, distribusi label, distribusi aspek, dan korelasi antar label.")

    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric("Jumlah Review", len(filtered_browse))

    with col2:
        avg_words = round(filtered_browse["word_count"].mean(), 2) if len(filtered_browse) > 0 else 0
        st.metric("Rata-rata Kata", avg_words)

    with col3:
        median_words = round(filtered_browse["word_count"].median(), 2) if len(filtered_browse) > 0 else 0
        st.metric("Median Kata", median_words)

    with col4:
        max_words = int(filtered_browse["word_count"].max()) if len(filtered_browse) > 0 else 0
        st.metric("Maksimum Kata", max_words)

    st.markdown("### Statistik Panjang Review")
    st.dataframe(
        filtered_browse["word_count"]
        .describe()
        .rename("nilai")
        .rename_axis("statistik")
        .reset_index(),
        use_container_width=True
    )
```

```
st.markdown("### Distribusi Panjang Review")
plot_hist(
    filtered_browse["word_count"],
    "Distribusi Panjang Review Berdasarkan Jumlah Kata",
    "Jumlah Kata",
    "Jumlah Review"
)

st.markdown("### Distribusi Label ABSA")
label_counts = filtered_absa["accept"].explode().value_counts()

if len(label_counts) > 0:
    st.dataframe(count_table(label_counts, "label"), use_container_width=True)
    plot_bar(label_counts, "Distribusi Label ABSA", "Jumlah Review", "Label")
else:
    st.warning("Tidak ada label yang sesuai dengan filter.")

st.markdown("### Distribusi Data per Aspek")

aspect_list = []

for labels in filtered_absa["accept"]:
    if isinstance(labels, list):
        for label in labels:
            aspect_list.append(get_aspect(label))

aspect_counts = pd.Series(aspect_list).value_counts()

if len(aspect_counts) > 0:
    st.dataframe(count_table(aspect_counts, "aspek"), use_container_width=True)
    plot_bar(aspect_counts, "Distribusi Data per Aspek", "Jumlah", "Aspek")
else:
    st.warning("Tidak ada aspek yang sesuai dengan filter.")

st.markdown("### Distribusi Entitas ABSA")

filtered_text_keys = set(filtered_absa["text_key"].tolist())
```

```
entity_filtered = entity_df.copy()
entity_filtered["text_key"] = entity_filtered["review_text"].apply(normalize_text)
entity_filtered = entity_filtered[entity_filtered["text_key"].isin(filtered_text_keys)]

if len(entity_filtered) > 0:
    entity_counts = entity_filtered["entity_label"].value_counts()
    st.dataframe(count_table(entity_counts, "entity_label"), use_container_width=True)
    plot_bar(entity_counts, "Distribusi Entitas ABSA", "Jumlah Entitas", "Label Entitas")
else:
    st.warning("Tidak ada entitas yang sesuai dengan filter.")

st.markdown("### Matriks Korelasi Antar Label")

if len(filtered_absa) > 0:
    label_binary = pd.DataFrame(0, index=filtered_absa.index, columns=LABELS)

    for idx, labels in filtered_absa["accept"].items():
        if isinstance(labels, list):
            for label in labels:
                if label in label_binary.columns:
                    label_binary.loc[idx, label] = 1

    label_binary = label_binary.loc[:, label_binary.sum(axis=0) > 0]

    if label_binary.shape[1] >= 2:
        corr_matrix = label_binary.corr()

        fig, ax = plt.subplots(figsize=(12, 10))
        im = ax.imshow(corr_matrix)
        fig.colorbar(im, ax=ax)

        ax.set_xticks(range(len(corr_matrix.columns)))
        ax.set_yticks(range(len(corr_matrix.columns)))

        ax.set_xticklabels(corr_matrix.columns, rotation=90)
        ax.set_yticklabels(corr_matrix.columns)
```

```
            ax.set_title("Matriks Korelasi Antar Label ABSA")

            plt.tight_layout()
            st.pyplot(fig)

            st.dataframe(corr_matrix, use_container_width=True)
        else:
            st.warning("Label yang tersedia setelah filter kurang dari dua, sehingga korelasi tidak dapat dihitung.")
    else:
        st.warning("Tidak ada data anotasi yang sesuai dengan filter.")


# ==========================
# HALAMAN IRR REPORT
# ==========================

elif page == "IRR Report":
    st.title("🤝 Inter-Annotator Agreement / IRR Report")
    st.write(
        "Halaman ini menampilkan hasil perhitungan Inter-Annotator Agreement menggunakan Cohen's Kappa "
        "berdasarkan file hasil anotasi."
    )

    st.markdown("### Ringkasan Data IRR")

    total_annotations = len(df_annot_raw)
    unique_reviews = df_annot_raw["_input_hash"].nunique() if "_input_hash" in df_annot_raw.columns else len(df_annot_raw)
    paired_reviews = get_paired_review_count(df_annot_raw)

    annotator_count = (
        df_annot_raw["_annotator_id"].nunique()
        if "_annotator_id" in df_annot_raw.columns
        else 0
    )
```

```
    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric("Total Baris Anotasi", total_annotations)

    with col2:
        st.metric("Review Unik", unique_reviews)

    with col3:
        st.metric("Jumlah Annotator", annotator_count)

    with col4:
        st.metric("Review Berpasangan", paired_reviews)

    if "_annotator_id" in df_annot_raw.columns:
        st.markdown("### Jumlah Data per Annotator")
        annotator_counts = df_annot_raw["_annotator_id"].value_counts()
        st.dataframe(count_table(annotator_counts, "annotator"), use_container_width=True)

    st.markdown("---")

    st.markdown("## Nilai Cohen's Kappa per Aspek")

    if len(kappa_aspect_df) > 0:
        display_aspect_df = kappa_aspect_df.copy()
        display_aspect_df["cohen_kappa"] = display_aspect_df["cohen_kappa"].round(3)
        display_aspect_df["percent_agreement"] =
display_aspect_df["percent_agreement"].round(2)

        st.dataframe(display_aspect_df, use_container_width=True)
        plot_kappa_bar(kappa_aspect_df, "aspek", "cohen_kappa", "Cohen's Kappa per
Aspek")
    else:
        st.warning("Nilai Kappa per aspek tidak dapat dihitung.")

    st.markdown("## Nilai Cohen's Kappa per Label")
```

```
    if len(kappa_label_df) > 0:
        display_label_df = kappa_label_df.copy()
        display_label_df["cohen_kappa"] = display_label_df["cohen_kappa"].round(3)
        display_label_df["percent_agreement"] =
display_label_df["percent_agreement"].round(2)

        st.dataframe(display_label_df, use_container_width=True)
        plot_kappa_bar(kappa_label_df, "label", "cohen_kappa", "Cohen's Kappa per Label")
    else:
        st.warning("Nilai Kappa per label tidak dapat dihitung.")

    st.markdown("---")

    st.markdown("## Interpretasi Hasil IRR")

    if len(kappa_aspect_df) > 0:
        best_aspect =
kappa_aspect_df.dropna(subset=["cohen_kappa"]).sort_values("cohen_kappa",
ascending=False).head(1)
        worst_aspect =
kappa_aspect_df.dropna(subset=["cohen_kappa"]).sort_values("cohen_kappa",
ascending=True).head(1)

        if len(best_aspect) > 0:
            best_name = best_aspect.iloc[0]["aspek"]
            best_value = best_aspect.iloc[0]["cohen_kappa"]
            best_interp = best_aspect.iloc[0]["interpretasi"]

            st.success(
                f"Aspek dengan kesepakatan tertinggi adalah **{best_name}** "
                f"dengan nilai Cohen's Kappa **{best_value:.3f}**, "
                f"yang termasuk kategori **{best_interp}**."
            )

        if len(worst_aspect) > 0:
            worst_name = worst_aspect.iloc[0]["aspek"]
            worst_value = worst_aspect.iloc[0]["cohen_kappa"]
            worst_interp = worst_aspect.iloc[0]["interpretasi"]
```

```
        st.warning(
            f"Aspek dengan kesepakatan terendah adalah **{worst_name}** "
            f"dengan nilai Cohen's Kappa **{worst_value:.3f}**, "
            f"yang termasuk kategori **{worst_interp}**."
        )

    st.markdown(
        """
        Secara umum, nilai Cohen's Kappa digunakan untuk melihat tingkat kesepakatan antar annotator
        setelah memperhitungkan kemungkinan kesepakatan yang terjadi secara kebetulan.

        Interpretasi yang digunakan:
        - Kappa < 0.00: sangat rendah atau tidak ada kesepakatan
        - 0.00 sampai 0.20: sangat rendah
        - 0.21 sampai 0.40: rendah
        - 0.41 sampai 0.60: sedang
        - 0.61 sampai 0.80: baik
        - 0.81 sampai 1.00: sangat baik
        """
    )

    st.markdown("---")

    st.markdown("## Analisis Penyebab Ketidaksepakatan")

    st.markdown(
        """
        Ketidaksepakatan antar annotator dapat terjadi karena beberapa alasan berikut:

        1. **Batas antar aspek belum selalu jelas.**
           Beberapa review dapat membahas lebih dari satu aspek secara bersamaan. Misalnya kata seperti
           "murah", "worth it", atau "terjangkau" dapat dianggap sebagai aspek `PRICE`,
           tetapi pada konteks tertentu juga dapat dikaitkan dengan `PROMOTION`.

        2. **Aspek Promotion cenderung ambigu.**
```

Tidak semua review menyebutkan promosi secara eksplisit seperti diskon, voucher, atau promo.

Akibatnya, annotator dapat berbeda dalam menentukan apakah sebuah review termasuk aspek promosi.

3. **Label OUT_OF_TOPIC dapat menimbulkan perbedaan interpretasi.**

Review yang membahas pelayanan, keramahan pegawai, pembayaran, atau pengalaman umum pelanggan

tidak selalu masuk secara jelas ke aspek Product, Price, Place, atau Promotion.

Hal ini dapat menyebabkan satu annotator memilih `OUT_OF_TOPIC`, sedangkan annotator lain memilih aspek lain.

4. **Perbedaan pemilihan multi-label.**

Karena satu review dapat memiliki lebih dari satu label, annotator bisa saja sepakat pada satu label,

tetapi berbeda pada label tambahan. Misalnya keduanya memilih `PRODUCT_POSITIVE`,

namun hanya satu annotator yang menambahkan `PLACE_POSITIVE`.

"""

)

st.markdown("### Contoh Review yang Tidak Disepakati Annotator")

if len(disagreement_df) > 0:

st.dataframe(disagreement_df, use_container_width=True, height=500)

else:

st.success("Tidak ditemukan contoh disagreement, atau data annotator berpasangan tidak tersedia.")